Parallel Performance Analysis of Ant Colony Optimization with OpenMP, MPI, and Hybrid Domain Decomposition

1st Caio Escorcia Lima Dourado
ENSTA Paris
Institut Polytechnique de Paris
Palaiseau, France
caio.escorcio@ensta-paris.fr

2nd Gustavo Loiola Dos Santos
ENSTA Paris
Institut Polytechnique de Paris
Palaiseau, France
gustavo.loiola@ensta-paris.fr

3rd Santiago Florido Gomez
ENSTA Paris
Institut Polytechnique de Paris
Palaiseau, France
santiago.florido@ensta-paris.fr

***Abstract*—This document analyzes the parallel execution of an Ant Colony Optimization simulation over a fractal terrain. The study compares a shared-memory OpenMP implementation, a distributed-memory MPI implementation based on subdomain decomposition with halo exchange and ant migration, and a hybrid MPI+OpenMP variant that introduces intra-subdomain thread-level parallelism. The reported results focus on iteration time, computation time, rendering cost, and communication overhead across multiple colony sizes in order to identify the practical scalability limits of each model and the tradeoffs between computation and communication.**

***Index Terms*—ant colony optimization, parallel computing, OpenMP, MPI, hybrid programming, domain decomposition**

I. Introduction

The Ant Colony Optimization problem is one of the most widely studied metaheuristics for the simulation of collective behaviors in nature. The implementation of the algorithm is proposed from the definition of multiple artificial ants, either as a class or from the vector-based description of their properties. Each of these artificial entities makes movement decisions based on a combination of local heuristic information and a shared collective memory, usually modeled through an artificial pheromone. This optimization approach was introduced by Marco Dorigo in the 1990s [1].

The importance of this optimization lies in its ability to address problems in which the search space grows rapidly and where exact methods face difficulties in terms of computational scalability. Nevertheless, in large-scale scenarios, not only because of the size of the ant colony but also due to the complexity of the map, limitations may arise in computation times and in the efficient handling of increasingly larger scales.

In this context, the imminent need emerges to incorporate scalability and parallelization strategies for ACO, aimed at reducing the execution time of each algorithm by decreasing the computational load and facilitating the treatment of even larger entities. For this reason, the present document addresses parallelization with shared memory through OpenMP, as well as two different distributed-memory parallelization strategies using MPI, with the objective of improving the handling of large instances in the description of the collective behavior of ants.

A. Simulation times measurement

The measured times are defined so as to separate the simulation into clearly differentiated phases, which makes it possible to identify the contribution of each operation to the overall computational cost. In `event_poll` (see Table I), the time spent handling SDL events is recorded, including window closing, basic interaction, and event-queue updates; this corresponds to interface overhead rather than algorithmic work.

In `ants_advance` (Table I), the behavioral core of the ants is measured, including movement decisions, neighboring pheromone reads, food transport, collection accounting, and pheromone marking. In `evaporation` (Table I), only the pheromone evaporation stage is measured, which is characterized by ordered memory traversal and conditional evaporation logic and is therefore favorable for locality- and vectorization-based optimization. In `update` (Table I), the consolidation of the pheromone state at the end of the step is included, together with buffer exchanges, restoration of special cells, reimposition of nest and food sources, and map maintenance. Finally, `advance_total` (Table I) provides the most representative measure of the simulation core cost because it integrates `ants_advance`, `evaporation`, and `update`.

In `render` (Table I), the time required to draw the scene is measured, including the `blit` stage and the full screen-frame representation cost. All these contributions are ultimately included in `iteration_total` (Table I), which represents the complete cost of one simulation iteration.

Additionally, general simulation information is stored, including the number of measured iterations, the final amount of food after the simulation ends, and the iteration at which the first food unit arrives.

B. Algorithmic Equivalence

A general methodology is proposed for the analysis of the general behavior of the parallelization strategies that

TABLE I
SUMMARY OF MEASURED EXECUTION TIMES IN THE SIMULATION.

Time metric	Brief description
`event_poll`	SDL event handling.
`ants_advance`	Ant movement and decisions.
`evaporation`	Pheromone evaporation.
`update`	State consolidation and restoration.
`advance_total`	Total simulation-core cost.
`render`	Frame rendering time.
`iteration_total`	Total iteration time.

are proposed to be implemented within the methodological framework of the stated problem. In the first place, a generalized execution and comparison framework is proposed, corresponding to the execution of the algorithm until the first food item reaches the nest and the 10 subsequent iterations. In this context, it is intended that the results provided by all the algorithms make it possible to identify the number of iterations at which this milestone is reached in the simulation, with the idea of verifying that this number is the same for ant colonies of the same size and, in this way, verifying that the simulated dynamics are in fact equivalent and therefore that their results are comparable.

It is also proposed to record the amount of food that reaches the nest during those 10 subsequent iterations as a sample of equivalence in the dynamics of the colony after the arrival of the first food item. Under this criterion, the common observation window is not defined only by a fixed number of iterations, but by a shared behavioral event in the simulation, which makes it possible to compare the different implementations from an equivalent point in the evolution of the colony. In this way, the comparison framework is intended not only to contrast execution times, but also to verify that the underlying simulated behavior remains consistent across the different parallelization strategies.

C. Vectorization application

The application of vectorization was carried out by replacing the old `ant.cpp` file, which contained a vector storing each `Ant` class (with its respective properties), with several separate vectors representing each of these properties. That is, the `seed`, `state`, and position (`x` and `y`) properties of each ant were transferred to four vectors within a larger class called `Population`, whose function is to manage the change in the ants' positions.

For this adaptation, the `advance` function was modified to receive (in addition to the pheromone and terrain conditions, etc.) the index of each ant in the `Population` class vectors. It maintains the previous logic but uses the name `advance_one` for a single ant and `advance_all` for all ants. Finally, for organizational purposes, other functions such as `add_ant` and `reserve` were created to add ants to the population and to initialize their vectors, respectively. Regarding changes in code execution, the ant-by-ant iterations (`ant.advance`) are now performed using `Population.advance_all` with the respective indices for every ant. Table II summarizes the main vectorized components.

TABLE II
SUMMARY OF VECTORIZED COMPONENTS IN THE POPULATION STRUCTURE.

Component	Brief description
`Population`	Class storing ant properties in separate vectors.
`Population::advance_one`	Advances a single ant using its vector index.
`Population::advance_all`	Iterates over all indices to advance the population.
`Population::add_ant`	Adds an ant to the population.
`Population::reserve`	Initializes and allocates memory for the vectors.

D. Shared-memory OpenMP parallelization

In the `population.cpp` implementation, shared-memory parallelization is introduced at the loop level over the ant population using OpenMP within the `advance_all` function. A `#pragma omp parallel` region is created, and the loop iterating over the ants is divided among the available threads using a `schedule(static)` clause, which evenly pre-assigns contiguous chunks of the population to each thread. To avoid data races and false sharing when ants update the pheromone grid, a two-phase update strategy is implemented. Instead of writing directly to the shared pheromone map, each thread maintains a dynamically resizing local buffer (`local_touched`) to record the flat indices of the cells traversed by its assigned ants. Concurrently, the amount of food collected by the ants is safely accumulated across all threads using an OpenMP `reduction(+ : food_delta)` clause. Once the parallel computation region concludes, the main thread sequentially iterates over all thread-local buffers to apply the marks to the global pheromone map. This strategy guarantees thread safety, avoids costly atomic locks during the movement phase, and preserves deterministic behavior in the simulation.

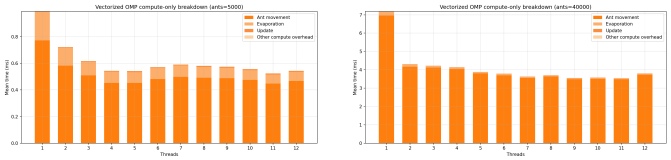
E. OpenMP results analysis

The dominant component in the execution time of the vectorized algorithm is the rendering process, which is not susceptible to being parallelized and therefore causes unexpected dynamics in the response of the system to the increase in the computing units. It is therefore proposed, for this section of the analysis, to analyze only the behavior of the headless execution of the algorithm, that is, without rendering, and to observe the effect that shared-memory parallelization has on the performance of the elements of the problem other than rendering.

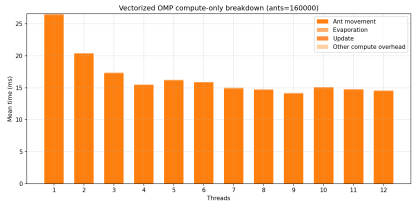
The performance of the vectorized OpenMP implementation can be analyzed through the execution-time breakdowns for different colony sizes, as illustrated in Fig. 1. The charts clearly demonstrate that the "Ant movement" phase completely dominates the computational cost across all configurations, regardless of the colony size. For small and medium thread

counts (from 1 up to approximately 4 or 6 threads), there is a noticeable reduction in the overall mean execution time, indicating a successful and effective parallel scaling.

However, as the number of threads continues to increase, the speedup tends to stagnate and, in some configurations, the total time even experiences a slight increase. This plateau effect is primarily caused by memory bandwidth saturation and the intrinsic overhead of OpenMP thread management. For smaller colonies, such as 5000 ants, dividing the workload among 8 to 12 threads results in very little useful computation per thread; consequently, the parallelization overhead becomes disproportionately large compared to the actual calculation time. While larger colonies (e.g., 40000 and 160000 ants) provide a sufficient workload to benefit more extensively from parallelization, they still suffer from diminishing returns at high thread counts due to shared-memory access contention. Furthermore, the non-parallelized sequential sections—such as the final consolidation of the pheromone updates and the constant "Evaporation" and "Update" stages—contribute to limiting the maximum achievable speedup, in accordance with Amdahl's Law.



(a) 5000 ants (b) 40000 ants



(c) 160000 ants

Fig. 1. Vectorized OpenMP compute-only breakdown for selected colony sizes.

II. Distributed-memory population decomposition

In this section, a distributed-memory parallelization paradigm using the Message Passing Interface (MPI) is explored. Unlike the domain-decomposition methods analyzed previously, this strategy maintains a fully replicated pheromone map on each MPI process while dividing the computational load associated with the entities. Specifically, the total ant population is partitioned equally among the P available MPI ranks (i.e., $N_{local} = N_{total}/P$). The ants independently traverse the entirely replicated fractal landscape and interact with their local copy of the pheromone grid.

During the initialization phase, the `MPI_Init`, `MPI_Comm_rank`, and `MPI_Comm_size` routines are utilized to determine the global execution context. Each rank uniquely seeds its random number generator and instantiates exactly its assigned subset of ants starting at the shared nest location.

A. Global pheromone synchronization

Because the ants evaluate movement constraints and deposit pheromones locally, the distributed pheromone maps inherently diverge at the end of the advancement phase. To maintain simulation correctness and consistency, these local maps must be merged into a single global map at the end of each iteration before the subsequent movement step is executed.

This synchronization is achieved using the `MPI_Allreduce` collective algorithm operating over the dense map buffers. When ants mark the terrain, the algorithm applies the `MPI_MAX` reduction operator. This mechanism ensures that if any process detects a newly laid pheromone trail, it propagates to all other processes, effectively computing the upper bound across the distributed grids. Similarly, following the parallel distributed evaporation stage—where each rank is responsible for evaporating a designated subset of the grid's rows—a secondary `MPI_Allreduce` utilizing the `MPI_MIN` operator merges the evaporated trails by capturing the smallest value across all ranks.

Finally, global simulation states, such as the iteration loop continuation flag and the total food collected, are synchronized via `MPI_Bcast` and `MPI_Allreduce` (with `MPI_SUM`), respectively. For rendering purposes, the visualization rank (rank 0) collects all scattered ant coordinates from the compute ranks using `MPI_Gather`.

B. Results analysis

For the performance analysis, it is more appropriate to focus on the headless version of the implementation, since it has already been observed that, for the ant-population sizes considered in this study, the overhead introduced by rendering is very high when compared with the cost associated with computation and communication. Moreover, rendering is not readily amenable to further parallelization, which means that its contribution can dominate the total execution time and mask the actual behavior of the numerical core of the algorithm.

For this reason, the evaluation of the proposed parallelization strategy is carried out using the version without rendering. This makes it possible to isolate the computational and communication costs that are directly associated with the algorithmic solution and, consequently, to obtain a more representative view of the real impact of parallelization on performance. In this way, the analysis is centered on the parts of the implementation that can actually benefit from distributed-memory optimization, avoiding distortions caused by a graphical stage whose cost is both significant and largely insensitive to the increase in computing resources.

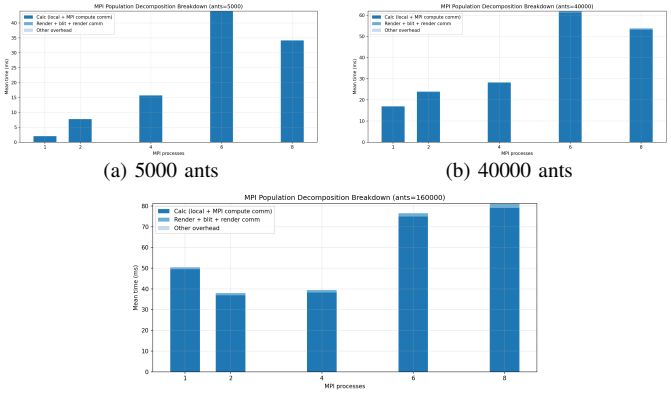
The performance of the MPI population decomposition can be evaluated through the total execution-time breakdowns illustrated in Fig. 2. It is immediately evident that while the ant movement (`ants_advance`) dominates the computational cost at low process counts, the communication overhead rapidly takes over as the dominant factor when more ranks are introduced.

When considering only the computational phase, the division of the ant population achieves a highly favorable strong scaling behavior. As demonstrated in Fig. 4 and Fig. 3, the workload dedicated to evaluating heuristic rules and updating ant positions decreases almost linearly with the number of processes. Because the ants act independently on their local maps, the numerical core balances perfectly without the need for complex ghost-cell boundary logic.

However, as the process count continues to increase, the overall speedup experiences a severe plateau and eventually degrades. This bottleneck arises directly from the communication overhead required for the global pheromone synchronization. The `MPI_Allreduce` collective operates on the entire 1024x1024 double-precision grid (approximately 8 MB per buffer). Executing two global reductions (`MPI_MAX` and `MPI_MIN`) every iteration demands substantial network bandwidth, transforming a constant-sized data merge into a severe scaling limitation.

The impact of this bottleneck is highly dependent on the problem size. For smaller colonies, such as 5000 ants, the computational workload is too light; the map synchronization cost dwarfs the ant computation almost immediately, leading to a disproportionately large parallelization overhead. Conversely, larger colonies (e.g., 40000 and 160000 ants) provide enough useful work to hide a portion of the communication overhead, pushing the scaling limit slightly higher before the network bandwidth is saturated.

Ultimately, this communication penalty effectively limits the theoretical acceleration predicted by Amdahl's Law. While entity-based workload division is conceptually simpler and computationally efficient, the associated cost of maintaining global map coherence makes it structurally unsuited for massive scale-out, resulting in a regime of diminishing returns that is reached much faster than in the localized subdomain decomposition model.



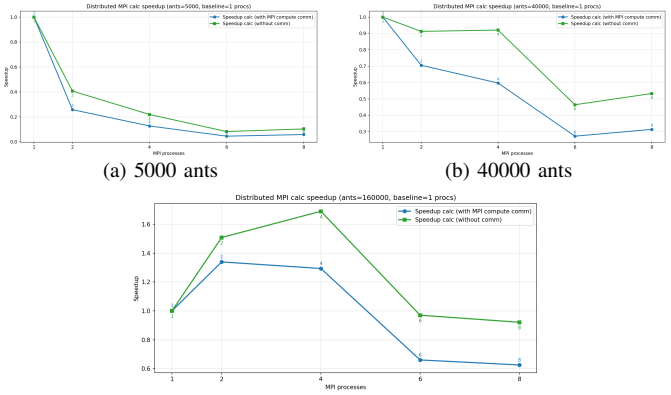
(a) 5000 ants (b) 40000 ants

(c) 160000 ants

Fig. 2. MPI population decomposition total iteration-time breakdown for selected colony sizes, showing communication overhead dominating at high process counts.

III. Distributed Subdomain MPI

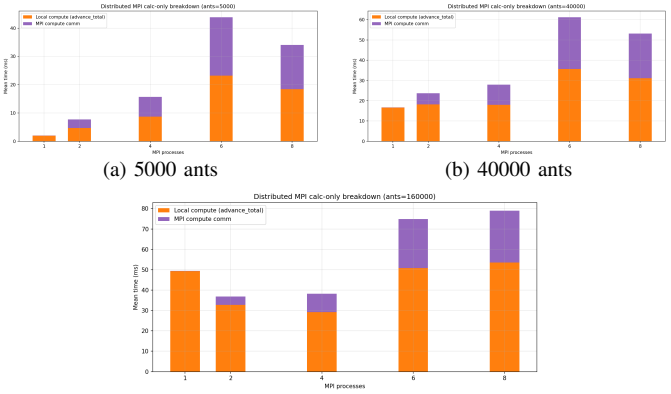
In this section, the implementation of an optimization strategy over the fractal domain is proposed based on the decomposition of the terrain into two-dimensional subdomains, which are managed by each MPI process. Each one is responsible for managing one of these subdomains in the map, considering the boundary effects between two-dimensional subdomains through the concept of ghost cells. Additionally, only the logic for generating a global terrain is preserved in the root node for data collection and subsequent rendering. In the rendered MPI runs, when more than one rank is available,



(a) 5000 ants (b) 40000 ants

(c) 160000 ants

Fig. 3. MPI population computation speedup (with and without communication) for selected colony sizes.



(a) 5000 ants (b) 40000 ants

(c) 160000 ants

Fig. 4. MPI population computation-only breakdown for selected colony sizes.

TABLE III
MPI COMPUTE SPEEDUP FOR ALL COLONY SIZES AND MPI PROCESS COUNTS.

Proc/Ants	5k	40k	160k
1	1.00	1.00	1.00
2	0.26	0.71	1.34
4	0.13	0.60	1.29
6	0.05	0.27	0.66
8	0.06	0.31	0.63

Note. The compute metric is $advance\_total + mpi\_halo\_exchange + mpi\_migration + mpi\_food\_allreduce$, matching the MPI compute plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each colony size, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

TABLE IV
MPI COMPUTE EFFICIENCY FOR ALL COLONY SIZES AND MPI PROCESS COUNTS.

Proc/Ants	5k	40k	160k
1	1.00	1.00	1.00
2	0.13	0.35	0.67
4	0.03	0.15	0.32
6	0.01	0.05	0.11
8	0.01	0.04	0.08

Note. The compute metric is $advance\_total + mpi\_halo\_exchange + mpi\_migration + mpi\_food\_allreduce$, matching the MPI compute plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each colony size, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

world rank 0 is reserved exclusively for visualization and is excluded from the Cartesian decomposition; therefore, the subdomains are assigned only to the compute ranks. In order to make that separation explicit, the implementation now keeps two communicators: a `compute_comm` for global reductions and rank-wide control among compute processes only, and a `cart_comm` for neighborhood exchanges tied to the Cartesian topology. The automated sweeps used for the plots now run headless by default; rendered executions are kept for exploratory runs, with the synchronous path as baseline and an optional `--async-render` flag for a decoupled render path.

The domain decomposition is constructed through a 2D MPI Cartesian topology. For this, `MPI_Dims_create` is used for the automatic factorization of the processes into a two-dimensional mesh of dimensions. Then, Cartesian communicators are created for the direct neighbors in the four directions: up, down, left, and right. In this way, each of the processes knows its Cartesian coordinates associated with the fractal terrain, as well as its local size and offset in both directions.

In addition to the interior block, each of the processes preserves a ghost-cell crown of thickness determined by parameterization in the case of the simulation, which is taken into account for local storage in the stride variables. The memory linear accesses as well as the indexations are performed considering the effect of those ghost cells that are implemented for the management of the neighborhood of the subdomains and the boundary conditions. It is this ghost-cell structure that makes it possible for MPI communication not to be required for each one of the ants that is located at the edge or boundary of its subdomain, considering that the algorithm only needs the immediate neighbors at a single-cell distance; therefore, a single layer of thickness 1 is sufficient to satisfy the algorithmic requirement.

The exchange of ghost cells for the fields that are static during the simulation, as in the case of the terrain and the cell type, is carried out with a classical one-halo-layer-per-boundary policy. Each process packs its first and last interior row for up/down, and its first and last interior column for left/right; then it launches non-blocking `MPI_Irecv` and `MPI_Isend` with the corresponding neighbors.

A. Pheromone system

The pheromone system is organized into two channels, each of which has its own memory buffer, which allows the distinction of two trails with a different semantics: an ant without load consults channel 0, while a loaded ant consults channel 1. To avoid coherence problems between the current pheromone state and the next one, two copies are preserved, one current and one next, so that all ants read the current version but write to the next version, and at the end of the step the exchange or swap is performed.

In terms of the exchange, instead of sending `v1` and `v2` separately between the subdomains, what is done is to pack them into a single interleaved buffer, for each one of the cells at the boundary of the subdomains. This provides a reduction in the number of messages to be sent with respect to independent fields, mainly because the communication costs exceed the cache-query costs associated with the buffer.

B. Ant migration

When an ant, as part of its movement, crosses a boundary and therefore the destination cell no longer belongs to the process to which this ant is assigned, a `TransitAnt` is constructed that contains the information associated with the properties of the ant, including the information for the pheromone update in the cell into which the ant enters before migration. The migration protocol is implemented with two neighborhood collectives over the Cartesian communicator. First, each process exchanges with its four neighbors the counts of outgoing ants per direction (up/down/left/right) by using `MPI_Neighbor_alltoall`. Then, with the received counts, it exchanges the payload arrays through `MPI_Neighbor_alltoallv` with `MPI_BYTE`, transmitting contiguous blocks of `TransitAnt` with size `count * sizeof(TransitAnt)`.

Finally, it concatenates what is received into an `active` vector. The ants that arrive at the process do not mean that they have already completed their step; that is why it calls `process_single_ant(...)` on each received active ant. That function continues the movement from the partial state of the ant; and if during that continuation it crosses another boundary again, it sends it again to another neighbor. The loop is not exited until the entire system remains "at rest" with respect to migrations, that is, until there is no ant pending to complete its distributed step. The global active-ant total is computed with `MPI_Iallreduce(..., MPI_SUM, ...)` and checked periodically (not every migration round), reducing global synchronization frequency while preserving the same completion criterion (`global_active == 0`).

C. Time measurement

Given the particular conditions of the fractal-terrain partitioning and the ownership of ants over two-dimensional subdomains, it is necessary to add new timing measurements in order to identify the computational costs of the strategies

implemented for parallelization with distributed memory. The time `mpi_halo_exchange` is included to measure the time taken by the halo exchange of pheromones in each iteration, which accounts for the communication time associated with the fixed neighborhood imposed by the spatial decomposition.

`mpi_migration` measures the time spent within `advance` in the exchange of ants between one rank and another, and is therefore responsible for measuring the cost associated with the distribution of the ant population across the subdomains handled by the processes. If the execution effectively has a single compute rank (for example, two world ranks with one dedicated only to rendering), that metric becomes zero because there is no distributed migration path to execute. For its part, the variable `mpi_food_allreduce` accounts for the blocking wait time associated with the non-blocking global food reduction (`MPI_Iallreduce` + `MPI_Wait`), after overlapping part of that communication with local computation.

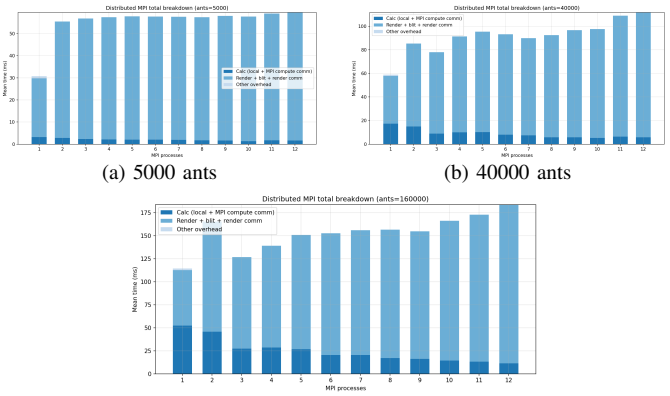
`mpi_render_comm` measures `gather_state_for_render(...)`, that is, the collection of the distributed state toward the render root in order to render pheromones and ants in a single global view. When a dedicated render rank is active, this metric also includes the relay of the assembled global frame from the compute root to world rank 0. In the default synchronous mode, each iteration waits for the render rank to finish the current frame before advancing, so the metric reflects the full communication cost required by the baseline rendered execution. When `--async-render` is enabled, the render path becomes asynchronous with respect to the numerical core: the render rank requests a frame when it is ready, and the compute side only gathers and relays the most recent state on demand. In that optional mode, `mpi_render_comm`, `event_poll`, `render`, and `blit` can be zero in iterations where no frame hand-off occurs, which reflects a dropped or deferred frame rather than missing simulation work. Finally, `halo_ms_local + migration_ms_local + food_allreduce_ms_local + render_comm_ms_local` are accumulated into `mpi_total_comm`, which accounts for the fraction of the total time spent in communication between MPI processes. Additionally, to reduce synchronization overhead in reporting, per-iteration timing reductions are aggregated and reduced in batches instead of reducing every metric in every iteration.

TABLE V
ADDITIONAL MPI COMMUNICATION TIMING METRICS.

Time metric	Brief description
`mpi_halo_exchange`	Pheromone halo exchange.
`mpi_migration`	Ant migration between ranks.
`mpi_food_allreduce`	Global food reduction.
`mpi_render_comm`	State gathering for rendering.
`mpi_total_comm`	Total MPI communication time.

D. Results analysis

It is therefore proposed to analyze the results of the MPI parallelization based on the subdivision into subdomains, and two elements become evident. The first is that the bottleneck in the execution of the simulations, both in the subdomain-based parallelization and in the other parallelization strategies that have been analyzed, lies in the graphical part of the algorithm as shown in Fig. 5. Even in the case in which the number of ants, and therefore the number of computational operations, grows up to 320000 ants in the colony, this causes the effect of the parallelization not to become necessarily evident in the analysis of the total time of each iteration in the simulation.In particular, the decomposition by components reveals that the computational stage preserves a favorable tendency with the increase in the number of processes, whereas the visual stage keeps a relatively high weight per iteration. This behavior explains why the global metric can suggest weak scaling even when the numerical core is effectively accelerated by the distributed scheme. Therefore, if the analysis is carried out only for the processing time and, consequently, the headless version of the algorithms is used in the comparison, implying that neither the `blit`, nor the rendering, nor the accumulation into a single process in the case of the subdivision by MPI domains is taken into account, the dynamics with respect to parallelization vary considerably. Additionally, it is worth emphasizing that, in the headless execution mode for the MPI subdomains, since there is no process dedicated exclusively to rendering, each of the processes is assigned a subdomain; this is why it is possible to use from 1 process onward in the comparison between parallelization methods.



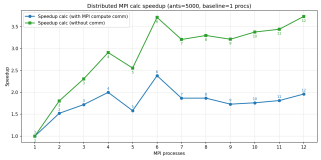
(a) 5000 ants (b) 40000 ants

(c) 160000 ants

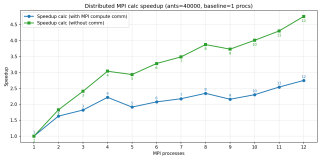
Fig. 5. MPI subdomain total iteration-time breakdown for selected colony sizes.

When the number of ants is small, as in the case of a colony of 5000 ants, it is possible to observe the effect produced by the incorporation of more processes in the acceleration of the computation, reaching a maximum close to 2 when communications between processes are considered. Nevertheless, when the speedup without communication is considered, a much more marked growth effect can be seen, reaching a speedup greater than 3 for 12 processes. However, this difference between the real speedup of the computation

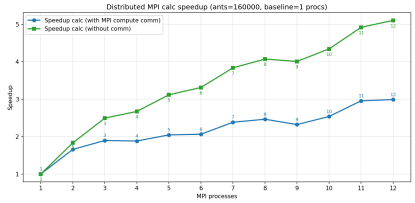
and the speedup without considering communication becomes increasingly larger as more subdomains are added in the representation of the fractal terrain and the communication between processes increases as depicted in Fig. 6 and Fig. 7. The increase in the size of the ant colony and the consequent increase in the number of performed computation operations account not only for a stable speedup dynamic as the colony size grows with respect to the number of processes, but also for increasing results, reaching speedups close to 3 for the complete computation and greater than 5 for the computation without communication in the best case with 12 processes, which is higher than the results obtained with small colonies. It is also important to clarify that, given the dynamics of the treatment of the problem, the percentage component of communication that intervenes in the computation becomes more and more representative as the number of processes increases, giving rise to a stagnation dynamic in the speedup for the configurations with the greatest number of processes.



(a) 5000 ants

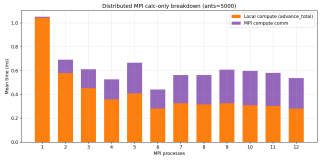


(b) 40000 ants

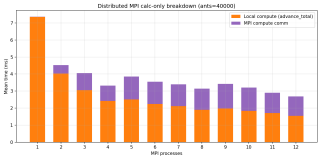


(c) 160000 ants

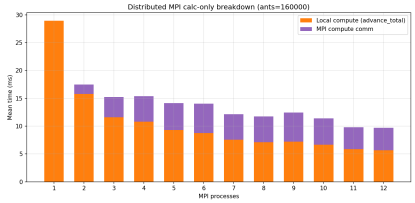
Fig. 6. MPI subdomain computation speedup (with and without communication) for selected colony sizes. The associated efficiency is computed with respect to compute MPI processes only, excluding rendering.



(a) 5000 ants



(b) 40000 ants



(c) 160000 ants

Fig. 7. MPI subdomain computation-only breakdown for selected colony sizes.

Nevertheless, it is evident that, unlike the case of parallelization with shared memory, there is an improvement in performance with the incorporation of processes which, although conditioned by the size of the buffers in which information is stored and also by the cost of communications, means that

TABLE VI
MPI COMPUTE SPEEDUP FOR ALL COLONY SIZES AND MPI PROCESS COUNTS.

Proc/Ants	5k	10k	20k	40k	80k	160k
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.52	1.52	2.01	1.63	1.69	1.66
3	1.72	1.80	1.94	1.82	1.84	1.90
4	2.00	1.87	2.67	2.22	2.42	1.88
5	1.58	1.54	2.14	1.91	1.96	2.05
6	2.38	1.60	2.44	2.07	2.06	2.06
7	1.87	1.76	2.70	2.17	2.41	2.38
8	1.87	1.76	2.70	2.34	2.42	2.46
9	1.73	1.80	2.61	2.16	2.35	2.32
10	1.76	1.90	2.83	2.30	2.49	2.54
11	1.81	2.11	3.05	2.54	2.78	2.96
12	1.96	2.23	3.07	2.75	2.86	2.99

Note. The compute metric is $advance\_total + mpi\_halo\_exchange + mpi\_migration + mpi\_food\_allreduce$, matching the MPI compute plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each colony size, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

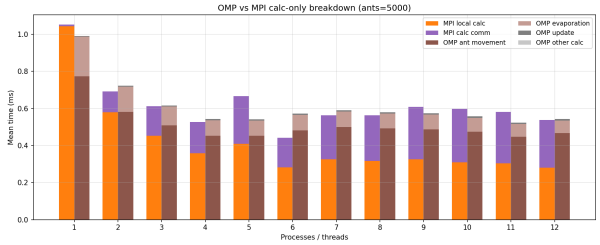
TABLE VII
MPI COMPUTE EFFICIENCY FOR ALL COLONY SIZES AND MPI PROCESS COUNTS.

Proc/Ants	5k	10k	20k	40k	80k	160k
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.76	0.76	1.01	0.81	0.85	0.83
3	0.57	0.60	0.65	0.61	0.61	0.63
4	0.50	0.47	0.67	0.55	0.61	0.47
5	0.32	0.31	0.43	0.38	0.39	0.41
6	0.40	0.27	0.41	0.35	0.34	0.34
7	0.27	0.25	0.39	0.31	0.34	0.34
8	0.23	0.22	0.34	0.29	0.30	0.31
9	0.19	0.20	0.29	0.24	0.26	0.26
10	0.18	0.19	0.28	0.23	0.25	0.25
11	0.16	0.19	0.28	0.23	0.25	0.27
12	0.16	0.19	0.26	0.23	0.24	0.25

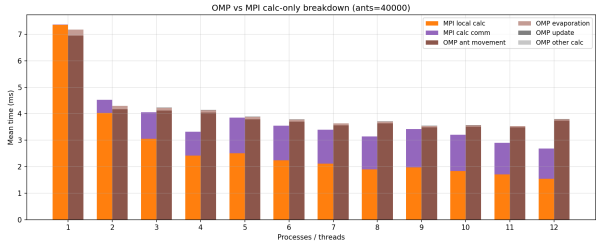
Note. The compute metric is $advance\_total + mpi\_halo\_exchange + mpi\_migration + mpi\_food\_allreduce$, matching the MPI compute plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each colony size, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

from the incorporation of more than one computation thread onward, the MPI parallelization shows, over the vast majority of the domain, a better performance per iteration than the OpenMP parallelization, as can be seen in Fig. 8.

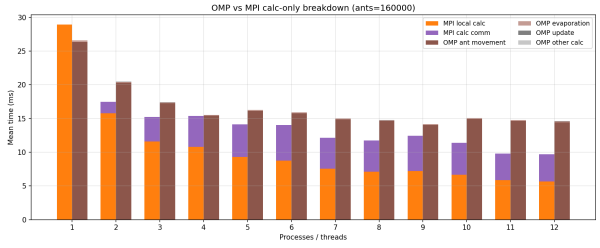
A relevant conclusion is that, as discussed, the scaling of the subdomain strategy is jointly constrained by the growth of communication overhead and the increasing memory-traffic pressure associated with migration buffers as both the process count and colony size rise. In practical terms, this means



(a) 5000 ants

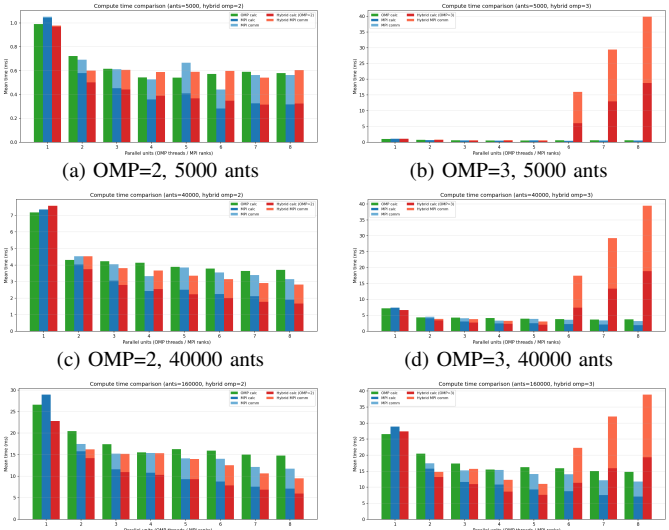


(b) 40000 ants



(c) 160000 ants

Fig. 8. MPI subdomain vs OpenMP computation-only breakdown for selected colony sizes.



(a) OMP=2, 5000 ants (b) OMP=3, 5000 ants

(c) OMP=2, 40000 ants (d) OMP=3, 40000 ants

(e) OMP=2, 160000 ants (f) OMP=3, 160000 ants

Fig. 9. Hybrid OMP-only vs MPI-only vs MPI+OpenMP computation-only comparison for selected colony sizes. The left column corresponds to two OpenMP threads per subdomain and the right column corresponds to three OpenMP threads per subdomain. The associated efficiency is computed with respect to compute MPI processes only, excluding rendering.

that the benefits of parallelization do not grow indefinitely, since a larger number of processes also introduces additional coordination costs and greater pressure on data movement. Consequently, although the inclusion of more processes improves performance over a broad operating region, the incremental benefit provided by each additional process becomes progressively smaller at high process counts. This behavior suggests that the system enters a regime of diminishing returns, where the computational advantages of distributing the workload are partially offset by communication and memory-access bottlenecks. This interpretation is consistent with the observed gap between the speedup with communication and the speedup without communication, which highlights the non-negligible impact of these overheads on overall scalability. Therefore, the results indicate that the efficiency of the subdomain strategy depends not only on increasing parallel resources, but also on controlling the communication patterns and memory demands that intensify as the problem size and level of parallelism grow.

E. Hybrid distributed subdomain

Considering the particularities of the solution implemented over the subdomains, the implementation of a hybrid model for the ant colonies is proposed, given that the cost of increasing the number of subdomains using MPI parallelization became evident, but it also became evident that the increase in subdomains causes a considerable increase in the number of communication operations between subdomains. Within this framework, hybrid integration is proposed as a solution so that each one of the subdomains has threads that execute in parallel with shared memory using OpenMP, as contrasted in Fig. 9. The objective is not necessarily to reengineer the algorithm, but rather to integrate shared-memory threads within each subdomain in order to increase parallelism in the computation without incurring the already mentioned costly communication costs associated with an increase in the number of subdomains into which the fractal terrain is decomposed. As in the MPI-only implementation, the automated hybrid sweeps used for these plots now run headless by default, while rendered runs remain available for exploratory testing.

TABLE VIII
HYBRID MPI+OPENMP COMPUTE SPEEDUP FOR OMP=2 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.63	1.44	1.47	1.68	1.74	1.41
3	1.61	1.50	1.71	1.99	1.60	1.50
4	1.66	1.50	1.76	2.07	1.79	1.49
5	1.66	1.55	1.93	2.26	1.78	1.63
6	1.64	1.69	1.74	2.41	2.02	1.82
7	1.81	1.89	1.89	2.61	2.47	2.14
8	1.62	1.91	2.06	2.69	2.03	2.40

Note. The compute metric is $advance\_total + mpi\_halo\_exchange + mpi\_migration + mpi\_food\_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports speedup $S = T(p_0)/T(p)$.

From the results of the hybrid implementation, it becomes

TABLE IX
HYBRID MPI+OPENMP COMPUTE EFFICIENCY FOR OMP=2 THREADS PER RANK.

Proc/Ants	5k	10k	20k	40k	80k	160k
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.81	0.72	0.74	0.84	0.87	0.70
3	0.54	0.50	0.57	0.66	0.53	0.50
4	0.42	0.38	0.44	0.52	0.45	0.37
5	0.33	0.31	0.39	0.45	0.36	0.33
6	0.27	0.28	0.29	0.40	0.34	0.30
7	0.26	0.27	0.27	0.37	0.35	0.31
8	0.20	0.24	0.26	0.34	0.25	0.30

Note. The compute metric is $advance\_total + mpi\_halo\_exchange + mpi\_migration + mpi\_food\_allreduce$, matching the hybrid sweep plots. Efficiency is computed with respect to the number of compute MPI processes only; rendering is excluded from the process count. For each combination of colony size and OpenMP threads per rank, the baseline is the smallest available MPI process count p_0. Each cell reports efficiency $E = S/(p/p_0)$.

evident that when two threads are implemented for each process, the computation execution-time results improve according to the number of processes, in most cases even more when the number of ants is larger and the increase in the parallel computing units favors the reduction of computation time. When the loads are medium or large, however, it is still evident that the fragmentation of the domain continues to cause significant increases in the time associated with migration, which means that although the computation time improves, the improvement is not sufficiently significant with respect to MPI parallelization when the total computation time is considered, as can be seen in the results for two threads per subdomain, which is the number of threads per subdomain that showed the best result in the test simulations.

It is also worth highlighting that when the number of MPI processes is increased and, in addition, the number of threads per rank is increased, each thread receives very few ants and the cost of coordinating those threads begins to consume an important part of the computation time, as can be seen in Fig. 9. For this reason, in the implementation of the hybrid model, too many threads per process become inefficient when the local subdomain is no longer sufficiently large, because the useful work per thread is not sufficiently significant and the cost of coordination and distribution of the results grows disproportionately with respect to the performance gain that it provides in the computation.

REFERENCES

[1] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics—Part B: Cybernetics*, vol. 26, no. 1, pp. 29–41, Feb. 1996.